

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



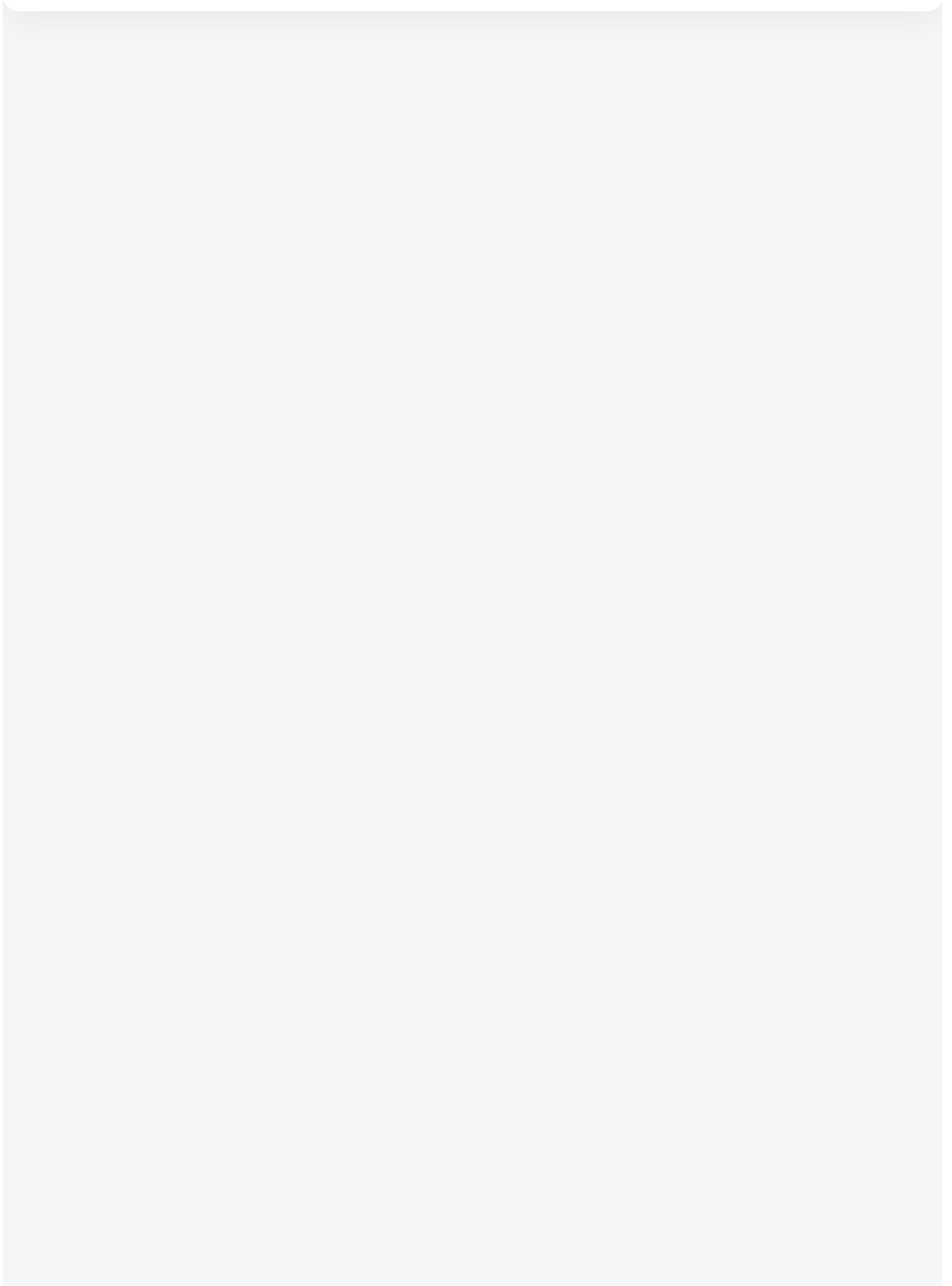
Jenkins, Maven, Tomcat et CI/CD

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**



Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

1. Objectifs et contexte

L'objectif est de construire une chaîne CI/CD pédagogique permettant de récupérer une application Java depuis Git, de la compiler avec Maven, d'exécuter les tests puis de déployer l'artefact WAR sur Apache Tomcat avec Jenkins.

- Comprendre le rôle du contrôleur Jenkins et de ses agents.
- Automatiser build et tests Maven.
- Produire un artefact reproductible.
- Déployer l'application sur Tomcat.
- Versionner le pipeline dans Git.

CI/CD : la CI valide le code par compilation et tests ; le CD automatise la livraison vers l'environnement cible.

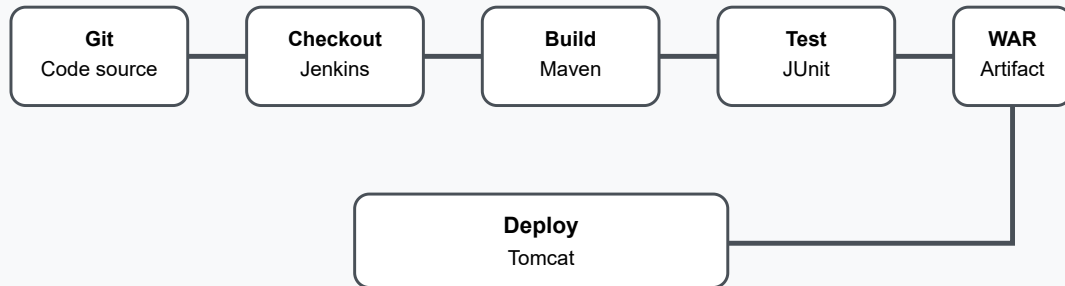
2. Prérequis

- Ubuntu 24.04 LTS ou Debian récent.
- Java 21 pour Jenkins.
- Accès réseau aux dépôts Jenkins, Maven et Git.
- Projet Java/Maven avec `pom.xml`.
- Serveur Tomcat accessible depuis l'agent Jenkins.
- Ports adaptés au laboratoire : 8080 pour Jenkins/Tomcat ; en production, privilégier HTTPS via reverse proxy.

Pour une petite équipe, Jenkins recommande environ 4 Go de RAM et 50 Go de stockage.

3. Concepts et architecture

Pipeline CI/CD Jenkins



Composant	Rôle
Git	Code source.
Jenkins Controller	Orchestration des pipelines et gestion des credentials.
Agent Jenkins	Exécution des builds.
Maven	Compilation, tests et génération du WAR.
Tomcat	Exécution de l'application web Java.
Artifact	Livable tel que <code>target/app.war</code> .

Développeur → Git → Jenkins Controller → Agent → Maven → Tests → WAR → Tomcat

Architecture : utiliser des agents dédiés pour les builds plutôt que le contrôleur lorsque le projet devient significatif.

4. Installation et configuration

4.1 Java 21

```
sudo apt update sudo apt install -y fontconfig openjdk-21-jre java -version
```

4.2 Jenkins LTS

```
sudo mkdir -p /etc/apt/keyrings sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc  
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key echo "deb [signed-  
by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable  
binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null sudo apt update  
sudo apt install -y jenkins sudo systemctl enable --now jenkins sudo systemctl status  
jenkins
```

Le paquet Jenkins utilise systemd et écoute par défaut sur 8080. Le secret initial est récupérable avec :

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

4.3 Maven

```
sudo apt install -y maven mvn --version
```

La version des dépôts peut varier. Maven 3.9.16 est actuellement une référence stable ; Jenkins peut également gérer Maven via **Manage Jenkins** → **Tools**.

4.4 Tomcat

```
sudo apt install -y tomcat10 sudo systemctl enable --now tomcat10 sudo systemctl  
status tomcat10 curl -I http://localhost:8080
```

Tomcat 10.1 est une branche stable. Une application ancienne basée sur `javax.*` peut nécessiter une migration vers `jakarta.*`.

5. Réalisation pratique

5.1 Build Maven local

```
mvn -B clean test package ls -lh target/*.war
```

5.2 Pipeline Jenkins

Créer un pipeline relié au dépôt Git puis ajouter le `Jenkinsfile` à la racine :

```
pipeline { agent any stages { stage('Checkout') { steps { checkout scm } }  
stage('Build') { steps { sh 'mvn -B clean package' } } stage('Test') { steps { sh  
'mvn -B test' } post { always { junit '**/target/surefire-reports/*.xml' } } }  
stage('Archive') { steps { archiveArtifacts artifacts: 'target/*.war', fingerprint:  
true } } stage('Deploy') { when { branch 'main' } steps { sh 'scp target/*.war  
deploy@TOMCAT_HOST:/tmp/app.war' sh 'ssh deploy@TOMCAT_HOST "sudo mv /tmp/app.war  
/var/lib/tomcat10/webapps/app.war"' } } } post { always { deleteDir() } } }
```

En pratique, utiliser Jenkins Credentials et des variables d'environnement pour l'hôte, la clé SSH et les secrets.

6. Commandes et explications

Commande	Utilité
<code>systemctl status jenkins</code>	État du service.
<code>journalctl -u jenkins -n 100</code>	Logs Jenkins.
<code>ss -lntp grep 8080</code>	Processus à l'écoute.
<code>mvn -B clean package</code>	Build reproductible.
<code>mvn -B test</code>	Tests unitaires.
<code>curl -I http://localhost:8080</code>	Test HTTP Tomcat.
<code>scp target/*.war deploy@HOST:/tmp/</code>	Transfert de l'artefact.

7. Vérifications et tests

1. `java -version`.
2. `systemctl is-active jenkins`.
3. `mvn --version`.
4. `systemctl is-active tomcat10`.
5. `curl -I http://localhost:8080`.
6. Exécuter le pipeline et vérifier Checkout, Build, Test, Archive et Deploy.
7. Tester l'application avec `curl -I http://TOMCAT_HOST:8080/app/`.
8. Vérifier les rapports JUnit et le WAR archivé.

Critère de succès : une erreur de compilation ou de test doit bloquer le déploiement.

8. Dépannage

Symptôme	Diagnostic	Action
Jenkins ne démarre pas	<code>journalctl -u jenkins -n 100</code>	Vérifier Java, mémoire et systemd.
8080 occupé	<code>sudo ss -lntp grep :8080</code>	Identifier le processus ou changer le port.
Build Maven échoue	Console Jenkins / <code>mvn -e</code>	Vérifier JDK, dépendances et <code>pom.xml</code> .
Tests rouges	Rapports Surefire/JUnit	Corriger les tests avant déploiement.
SSH/SCP échoue	<code>ssh -vvv deploy@HOST</code>	Vérifier clé, utilisateur, réseau et permissions.
Tomcat ne déploie pas	<code>journalctl -u tomcat10</code>	Vérifier WAR, Java et permissions.

9. Sécurité et bonnes pratiques

- Utiliser Jenkins LTS et maintenir les plugins à jour.
- Protéger Jenkins par HTTPS, authentification forte et filtrage réseau.
- Stocker clés, tokens et mots de passe dans Jenkins Credentials.
- Utiliser un compte de déploiement dédié avec privilèges minimaux.
- Éviter un accès sudo généraliste pour le compte de déploiement.
- Utiliser des agents isolés pour les builds.
- Limiter la conservation des anciens builds et artefacts.
- Déployer uniquement après validation des tests.
- Ne pas exposer le Tomcat Manager publiquement.
- Sauvegarder la configuration Jenkins et surveiller les logs.

10. Checklist

- ☐ Java 21 installé
- ☐ Jenkins LTS actif
- ☐ Accès Jenkins sécurisé
- ☐ Git configuré
- ☐ Maven configuré
- ☐ Tomcat actif
- ☐ Projet Maven compilable
- ☐ Tests intégrés
- ☐ Jenkinsfile versionné
- ☐ Credentials configurés
- ☐ WAR archivé
- ☐ Déploiement testé
- ☐ Logs et sauvegardes vérifiés

11. Conclusion

La chaîne CI/CD associe Git, Jenkins, Maven et Tomcat afin d'automatiser le cycle de livraison d'une application Java. Le pipeline devient reproductible, auditable et contrôlable.

Pour une plateforme de production, compléter cette base par des agents isolés, HTTPS, gestion stricte des secrets, contrôles de qualité, rollback, observabilité et déploiement progressif.

